The Drift Complexity Index: Matching Detector Resolution to Workload Drift

Arun Reungsilpkolkarn
arun.r@bu.ac.th
School of Information Technology and Innovation, Bangkok University
Pathum Thani, Thailand

Abstract

Workload drift—the motion of a workload's feature vector—has a measurable *complexity*: the effective number of independent modes the motion occupies. We introduce the *Drift Complexity Index* (DCI), a participation ratio of the drift-motion covariance, computable in $O(D^2)$ without eigendecomposition. Under a standard detection model, DCI bounds how much of the multi-dimensional detector's power the best single-axis detector retains: below one frontier a cheap detector is sufficient within a derived tolerance; above a second it is insufficient. Between the two, the spectrum decides. Empirically, ordinary drift is *near* rank-one; the regime is a property of the drift, not of the kernel. Measured feature cost concentrates in one axis—the positional kernel, 94% of extraction time. The cheap tier is therefore a Bonferroni union of the four cheap axes, and a two-statistic router pays for positional resolution only when drift is present and no single axis carries it. Live on PostgreSQL and MongoDB, the routed detector catches every template and volume onset, halves the full detector's false alarms, and costs 5.5–7.6× less, measured. A fixed single-direction detector misses heterogeneous drift: recall 0.60, stale indexes at 10.9× latency. DCI is a monitoring-resolution primitive for self-driving data systems.

Keywords

workload drift, drift detection, index advisor, workload similarity, complexity index

1 Introduction

Modern databases ship index advisors that recommend a physical design for a workload, but an advisor is expensive to run: it searches a large space of hypothetical indexes, often estimating costs for thousands of candidates. Running one on every window of a long-lived workload is wasteful — most of the time the workload has not changed enough to warrant new advice. The standard remedy is to *gate* the advisor: invoke it only when the workload *drifts.* Gating in turn needs a drift detector that — unlike the advisor it gates — runs on *every* window. It is the always-on monitoring layer.

How elaborate must that layer be? A rich, multi-dimensional workload-similarity kernel is safe — it sees drift along any axis — but it is expensive to evaluate every window. A single cheap similarity dimension is the opposite: nearly free, but blind to drift that does not move that axis. Choosing between them blindly is unsatisfying. This paper asks whether one can tell, cheaply and *before* committing to a detector, how much detection machinery a workload's drift requires.

Our answer is that workload drift has a measurable *complexity.* Drift is the motion of a workload's feature vector; the number of independent modes that motion occupies is a workload property, estimated at runtime with the *Drift Complexity Index* (DCI) — a participation ratio of the drift-motion covariance, computable in $O(D^2)$ with no eigendecomposition. DCI is revealing. Ordinary workload drift is *near* rank-one: a feature launch or a traffic change moves the similarity axes together, so a single dimension already captures it. Genuinely complex drift requires *heterogeneous* change — several kinds of drift at once — and only then is the full kernel needed. Because DCI separates these regimes *before* any detector runs, the monitoring layer can be placed at the cheapest sufficient resolution. We treat DCI not as a one-off gating heuristic but as a general primitive—a cheap, engine-portable measure of *how* a workload drifts—whose first application is deciding a detector's resolution. The layer it builds is deliberately *advisor- and engine-agnostic*: it reads only query text and arrival timestamps and treats the advisor as a black box, so the same construction runs unchanged in front of any advisor, on any engine whose workload yields a per-window feature vector—validated on a relational and a document store.

This paper makes four contributions. **(C1)** The Drift Complexity Index: a cheap, bounded, engine-portable *primitive* for workload-drift complexity, with the eigenvalue bounds that tie it to detector sufficiency (§4). **(C2)** A *regime map*: across three SQL workloads and a document store (MongoDB), a single similarity axis suffices below a DCI frontier near 1.5 and the full kernel is needed above it (§6); the same regime is reproduced by kernel-free feature representations and appears on a real workload trace (§6.9). **(C3)** A two-tier monitoring layer placed by the measured cost structure — a Bonferroni union of the cheap axes, and a two-statistic router that buys positional resolution only when drift is present and no axis carries it — with a derived, verified sufficiency tolerance; live on two engines it matches the full detector's recall with half its false alarms at 5.5–7.6× lower measured cost (§5, §6). **(C4)** A practitioner rule, validated end-to-end: profile a workload's DCI and deploy the matching detector; a compact deployment on live PostgreSQL and MongoDB (§6.10) confirms the routed advisor closes the loop in both regimes — on mixed drift it matches the always-multi-D detector's decisions where a fixed single-direction detector structurally cannot. The full economic value of gating — throughput, plan quality — composes with but is separable from ours, and is left to future work (§7).

2 Related Work

Index advisors — the gated component. The advisor DCI gates is itself a deep and active line of work. Cost-driven index selection and the optimiser-coupled "what-if" interface [7, 8] matured into productised advisors that jointly choose indexes, views, and partitions [1] — shipping at cloud scale as auto-indexing services with fixed validation cadences [6, 9] — and into online, learning-based tuners: bandit formulations with regret-based safety guarantees [31, 32] and advisors for dynamic, heterogeneous workloads [40]. Recent work pushes on scale [4, 39], online

EDBT '27, Lille (France)

uncertainty [43], learned search over hybrid designs [29], and LLM-guided configuration [20]. An in-depth study consolidates the index-advisor design space [47], and the self-driving-DBMS programme frames the broader goal [30]. This machinery is increasingly *engine-portable*: cross-DBMS tuners span PostgreSQL and MySQL [37, 38], and cost-based physical design reaches NoSQL document and column stores [25]. All of this work decides *which* physical design to build, or *how* to search for it. DCI sits one layer above and is advisor-agnostic: it decides, cheaply and on every window, *how much detection machinery* a workload's drift warrants before the (expensive) advisor is consulted at all. Even a drift- or uncertainty-aware advisor [40, 43] must first be reached; DCI governs the detector that decides when to reach it.

Drift and change-point detection. The always-on layer DCI routes is a drift detector. Concept-drift detection — DDM [11], ADWIN [3], and the surveys that organise a still-growing field [2, 12, 23] — and the sequential change-point theory beneath it — CUSUM [28] and its multivariate extensions [14], window-limited GLR schemes with delay/false-alarm optimality [18, 19], and sketching that compresses the test statistic [5] — economics our setting inverts, since feature extraction rather than the statistic dominates monitoring cost (§6.3) — answer *whether* and *when* drift occurred, and a large recent literature continues to sharpen their accuracy and adaptivity [2]. DCI answers a prior, orthogonal question: *how complex* the drift is, and hence which detector to run at all. Lu et al. [23] name "drift understanding" as a task distinct from detection; DCI turns that qualitative notion into a concrete, $O(D^2)$ statistic that routes the monitoring layer; Webb et al. [41] quantify drift's magnitude and duration, and DCI adds the orthogonal *complexity* axis. In databases specifically, workload-shift detectors [16] instantiate the always-on layer DCI routes. That the cost of the always-on detector is itself a first-class concern is increasingly recognised [27], and drift is becoming a first-class *benchmarking* dimension: DriftBench generates data- and workload-drift for benchmark inputs [22] and NeurBench evaluates learned components under controlled drift factors [46] — the generation-side complements of DCI, which prices a given drift's complexity.

Hierarchical hypothesis testing. The nearest prior work is hierarchical hypothesis testing for drift [44, 45]: a cheap, always-on Layer-I test backed by an expensive Layer-II test invoked *after* a Layer-I alarm. That is *escalate-on-event* — the expensive machinery runs in response to a detected event. DCI is *select-on-complexity*: it picks the detector *before* any window is scored, from a stationary property of the workload's drift geometry. The two are orthogonal and composable — DCI sets the resolution of the always-on layer, and an escalation step could still confirm its alarms.

Effective dimensionality. DCI's core, the participation ratio, originates in condensed-matter physics as a count of effective modes [21] and appears in computational neuroscience as the effective dimensionality of neural covariance [13]; in statistics the same functional is the effective degrees of freedom of the Welch–Satterthwaite approximation [36, 42], and in signal processing the effective rank [35]. To our knowledge this is its first use to characterise *workload* drift and to route a database monitoring layer. It is likewise distinct from workload *characterization and forecasting*, which models what a workload *will be* [17, 24]: DCI measures a structural property of *how* a workload drifts.

3 Background: The HSM Workload-Similarity Kernel

The drift signal that DCI operates on comes from an HSM-style workload-similarity feature [34], which is *prior work*; we define the instantiation we use in full here only so the paper is self-contained. Our contributions — the Drift Complexity Index, its bounds (§4), and the DCI-routed monitoring layer (§5) — take the kernel as nothing more than a black-box source of the per-window feature vector, and apply to any other source of one. The specific five axes are one instantiation in the HSM tradition [34], not a contribution of this paper; §6.9 confirms DCI's regime is reproduced by a generic similarity that uses none of them. (That several axes are needed to *span* the drift a kernel can meet across workloads [34] is compatible with any one block's drift being near rank-one: spanning concerns the directions drift *can* take, DCI how many a given block's drift *does* occupy.)

HSM is a *pre-execution* measure: it reads a workload's query text and the arrival timestamps of its queries, and never executes a query, builds an index, or inspects an execution plan. A sliding *window* of queries is aggregated into a per-template frequency vector, a query count, the sets of tables and columns referenced, and a one-second-resolution arrival series. From a pair of windows the kernel computes five pairwise similarity scores, each in $[0,1]$ (1 = identical; Table 1), and a weighted composite. A window's feature vector $f = (S_R, S_V, S_T, S_A, S_P)$ is thus its similarity to a reference window.

Definition 1 (Window and feature vector)**.** A *window* W_t is the query text and arrival timestamps of a sliding block of queries. Fixing a reference window W_0, the kernel maps W_t to $f(t) = (S_R, S_V, S_T, S_A, S_P) \in [0,1]^5$, the five pairwise similarity scores of W_t against W_0 (Table 1), each equal to 1 at identity and decreasing with dissimilarity. *Workload drift* is the motion of $f(t)$ across t; every construct in this paper is a function of that trajectory.

For the rest of the paper, then, each window is simply a bounded, correlated feature vector $f \in [0,1]^5$, and that vector is all the detectors ever see. A single-axis detector tracks drift on one axis of f; the multi-dimensional detector tracks all five jointly — §5 fields them as a cheap union tier over the four inexpensive axes and a full tier — so the resolution question is simply how many of the five a workload's drift actually moves, and which axes carry it. Nothing downstream depends on the kernel *itself*: DCI is a function of the covariance of f's motion alone, so a plain per-template query-frequency vector or a learned query/plan embedding is an equally valid input; only the numerical frontier (§6) is feature-set-specific. This lets the *same* construction read a document store in §6.2: MongoDB aggregation-pipeline templates and their collections and fields take the place of SQL templates and tables and columns, while the five-axis vector is computed identically.

The five axes span distinct facets of a window pair — template ranking and direction (S_R, S_T), query volume (S_V), the tables and columns referenced (S_A), and the arrival-time shape (S_P); Table 1 gives their closed forms, and the composite weights are the fixed defaults of the HSM line [34].

Two kernel properties matter for what follows. First, the five axes are *correlated*: S_R and S_T are two readings of the same per-template frequency vector — a rank correlation and a cosine angle — and the measured Gram off-diagonals on a real workload run from 0.68 to 0.91. A detector that fuses the axes must therefore account for this covariance rather than treat them as independent

Table 1: The five similarity axes and the composite of the HSM-style feature used in our experiments [34]; each score is in $[0, 1]$.

Axis	Formula	Measures
S_R	$1 - \arccos(\rho_s)/\pi$	template rank corr.
S_V	$\min(Q_a, Q_b)/\max(Q_a, Q_b)$	query-count ratio
S_T	$1 - \frac{2}{\pi}\arccos(\hat{v}_a^\top \hat{v}_b)$	template cosine
S_A	$\frac{1}{2}J(T_a, T_b) + \frac{1}{2}J(C_a, C_b)$	table/column Jaccard
S_P	$\sum_{\beta=1}^{4} \lambda_\beta\, sc_\beta$	arrival-pattern shape
HSM	$.25S_R{+}.20S_V{+}.20S_T{+}.20S_A{+}.15S_P$	composite

(§5). Second, the composite HSM is a fixed weighted *average*; it therefore attenuates drift confined to a single axis, so we use it as a gating score, not a detector (§5).

4 The Drift Complexity Index

Drift is the *motion* of the feature vector f. Over a block of windows we form the $D \times D$ covariance C of the drift-motion deviations ($D = 5$): a symmetric positive-semidefinite matrix with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_D \geq 0$ and normalised spectrum $p_i = \lambda_i / \sum_j \lambda_j$. The *Drift Complexity Index* is

$$\mathrm{DCI} = \frac{\mathrm{tr}(C)^2}{\|C\|_F^2} = \frac{1}{\sum_i p_i^2}\,. \tag{1}$$

This is the *participation ratio* of the drift spectrum — equivalently the exponentiated order-2 Rényi entropy [33] or the Hill number of order two [15]: the effective number of independent modes the drift occupies.

DCI has the three properties a runtime index needs. It is *bounded* — DCI $\in [1, D]$, equal to 1 when the drift is rank-one and to D when its energy is spread evenly — so a threshold on it is interpretable. It is *cheap*: the closed form in (1) is a trace and a Frobenius norm, $O(D^2)$ with no eigendecomposition — the spectrum itself would cost $O(D^3)$. Measured on the pinned platform (identical values to 10^{-13}), the closed form costs 1.4 vs. the spectrum's 4.9 μs at this paper's D=5, and 59 vs. 6157 μs at D=500 — a 105× ratio that matters exactly where richer per-window features (raw frequency vectors, learned query embeddings; §6.9) would take DCI. Estimating the drift covariance over an N-window block adds an $O(ND^2)$ pass; but DCI is computed once per block to set the route, not per window like a detector, so this cost is amortized off the monitoring hot path. And it is *dimensionless*, a ratio of like-unit quantities, hence comparable across workloads and engines without calibration.

Two exact bounds on the dominant mode's share $p_1 = \max_i p_i$ ground the regime claim. Recall $p_i \geq 0$ and $\sum_i p_i = 1$, and write $S = \sum_i p_i^2 = \mathrm{DCI}^{-1}$.

Proposition 1 (low complexity forces a dominant mode). *$p_1 \geq \mathrm{DCI}^{-1}$, with equality iff every non-zero p_i equals p_1.*

Proof. Each $p_i \leq p_1$ and $p_i \geq 0$, so $p_i^2 = p_i \cdot p_i \leq p_i p_1$; summing over i, $S = \sum_i p_i^2 \leq p_1 \sum_i p_i = p_1$, i.e. $p_1 \geq \mathrm{DCI}^{-1}$. Equality forces $p_i \in \{0, p_1\}$ for every i. □

Proposition 2 (high complexity forbids one). *$p_1 \leq \mathrm{DCI}^{-1/2}$, with equality iff the drift is rank-one.*

Proof. Every term of S is non-negative, so $S = \sum_i p_i^2 = p_1^2 + \sum_{i \geq 2} p_i^2 \geq p_1^2$; hence $p_1 \leq \sqrt{S} = \mathrm{DCI}^{-1/2}$. Equality forces $p_i = 0$ for all $i \geq 2$. □

The two bounds bracket the dominant mode. As DCI $\to 1$, Proposition 1 drives $p_1 \to 1$: a single mode carries the drift and a one-dimensional detector captures it. As DCI grows, Proposition 2 drives p_1 down: no axis can carry the drift, so a one-dimensional detector must lose information. Both bounds are tight — attained at the flat-support and rank-one spectra. DCI therefore predicts, before any detector runs, which regime a workload's drift is in.

Proposition 3 (complexity counts concurrent drift causes). *If, over a block, the drift is driven by k mutually uncorrelated causes—$d(t) = \sum_{c=1}^{k} g_c(t)\, v_c$ with fixed directions v_c and uncorrelated zero-mean amplitudes g_c—then $C = \sum_c \mathrm{Var}(g_c)\, v_c v_c^\top$ has rank at most k and* DCI $\leq k$.

Proof. C is a sum of k rank-one PSD terms, so $\mathrm{rank}(C) \leq k$ and at most k of the p_i are non-zero. For a distribution on r non-zero atoms, $\sum_i p_i^2 \geq 1/r$ by Cauchy–Schwarz, so DCI $= 1/\sum_i p_i^2 \leq r \leq k$. □

Proposition 3 explains the regime map mechanistically: a single-cause change—a template shift *or* a volume change—gives k=1 and rank-one drift, whereas drift on two independent fronts gives k=2 and DCI $\leq$ 2. Measured values sit just under this ceiling (§6: single-cause cells at DCI $\approx$ 1, mixed at $\approx$ 2), the small excess being finite-sample noise. The two regimes are thus not an artifact of a particular generator but reflect how many independent causes drive the drift—a property of the workload, not the kernel.

4.1 A worked example

The bounds turn a single scalar into a resolution verdict. Under *template* drift on TPC-H the drift covariance is almost rank-one (DCI = 1.02); Proposition 1 guarantees $p_1 \geq 0.98$, so one axis carries $\geq$ 98% of the drift energy and the cheap 1-D detector suffices—indeed three axes are within tolerance of the full kernel. Under *mixed* drift (DCI = 1.97), Proposition 2 caps $p_1 \leq 0.71$, so no single axis tracks the drift and the multi-dimensional detector is required. At the frontier $\tau = 1.5$, Propositions 1–2 bracket $p_1 \in [0.67, 0.82]$—neither dominant (as at low DCI) nor forbidden (as at high DCI), the transition zone the empirical frontier occupies.

4.2 From spectrum to detector: a sufficiency guarantee

The bounds constrain the drift spectrum; we now connect that spectrum to what a detector can see. The intuition is one line: a linear detector's power is its signal-to-noise ratio along the direction it watches, so, averaged over random drifts, the best fixed direction captures exactly the dominant eigenvalue's share of the drift energy — p_1. Formally, model the steady state as $d \sim (0, \Sigma)$ and a drift as a mean shift s, random over a block with $s \sim (0, C)$; a linear detector along a unit direction a has deflection (squared

SNR) $\rho(a; s) = (a^\top s)^2/(a^\top \Sigma a)$, and the multi-dimensional detector realises the full non-centrality $\rho_{\text{full}} = s^\top \Sigma^{-1} s$.

Theorem 1 (dimensional deflection ratio). *Under white steady noise ($\Sigma \propto I$), averaged over drifts $s \sim (0, C)$, the best fixed 1-D detector's mean deflection relative to the full detector's mean non-centrality equals the dominant-mode share,*

$$R = \frac{\max_a \mathbb{E}_s[\rho(a; s)]}{\mathbb{E}_s[\rho_{\text{full}}]} = \frac{\lambda_1}{\sum_i \lambda_i} = p_1,$$

hence $\mathrm{DCI}^{-1} \le R \le \mathrm{DCI}^{-1/2}$ *by Propositions 1–2.*

Proof. Under $\Sigma \propto I$, $\mathbb{E}_s[\rho(a; s)] = (a^\top C a)/(\sigma^2 \|a\|^2)$, maximised at the top eigenvector of C with value λ_1/σ^2; and $\mathbb{E}_s[\rho_{\text{full}}] = \mathrm{tr}(C)/\sigma^2$. The ratio is $\lambda_1/\mathrm{tr}(C) = p_1$; the bounds are Propositions 1–2 on the spectrum of C. □

The identity is neutral—one direction captures a fraction *exactly* p_1 of the drift energy—and the routing regime follows as a corollary and its converse.

Corollary 1 (sufficiency domain). For $\delta \in (0, 1)$, on $\{\mathrm{DCI} \le 1/(1-\delta)\}$ the best 1-D detector retains $R = p_1 \ge \mathrm{DCI}^{-1} \ge 1 - \delta$ of the full non-centrality.

Proposition 4 (tightness at integer complexity). *For every integer $m \in \{2, \ldots, D\}$ there is a drift covariance with* $\mathrm{DCI} = m$ *for which the best 1-D detector retains only* $R = 1/m$*; the sufficiency guarantee of Corollary 1 is tight.*

Proof. Take $C = \mathrm{diag}(\frac{1}{m}, \ldots, \frac{1}{m}, 0, \ldots, 0)$ with m equal nonzero eigenvalues: $p_i = 1/m$, $\mathrm{DCI} = m$, $p_1 = 1/m$, so $R = 1/m$ by Theorem 1. (The restriction to integer m is forced: by Proposition 1, $R = 1/\mathrm{DCI}$ requires a flat spectrum, whose support size is an integer.) □

Corollary 2 (a sufficiency trichotomy). Fix $\delta \in (0, 1)$. If $\mathrm{DCI} \le (1-\delta)^{-1}$, the best 1-D detector is $(1-\delta)$-sufficient (Corollary 1). If $\mathrm{DCI} > (1-\delta)^{-2}$, no single direction is: $R = p_1 \le \mathrm{DCI}^{-1/2} < 1 - \delta$. In between, the bracket $[\mathrm{DCI}^{-1}, \mathrm{DCI}^{-1/2}]$ straddles $1 - \delta$, so the bounds alone do not decide: sufficiency is spectrum-dependent, with both behaviours realised.

Proof. The first zone is Corollary 1; the second follows from Proposition 2, since $\mathrm{DCI} > (1-\delta)^{-2} \iff \mathrm{DCI}^{-1/2} < 1 - \delta$. For the middle zone, both behaviours occur at $\delta = \frac{1}{3}$ (zone $(1.5, 2.25]$): the flat spectrum with $m = 2$ has $\mathrm{DCI} = 2$ and retains only $R = \frac{1}{2} < \frac{2}{3}$, while a spiked spectrum — $p_1 = 0.67$ with four equal tails of 0.0825 — has $\mathrm{DCI} = 2.10$ yet retains $R = p_1 = 0.67 > \frac{2}{3}$. □

The pieces compose into a floor for DCI-thresholded routing — the reference policy the regime map instantiates.

Corollary 3 (DCI-routing floor). Under DCI-thresholded routing at τ, every window-block is scored by a detector whose mean deflection is at least $\min(1, \tau^{-1})$ of the full non-centrality: blocks routed to the multi-dimensional detector realise it entirely, and blocks routed to the 1-D detector have $\mathrm{DCI} < \tau$, hence $R = p_1 \ge \mathrm{DCI}^{-1} > \tau^{-1}$ by Proposition 1 and Theorem 1.

At $\tau = 1.5$ this reference policy never operates below two-thirds of the full detector's mean deflection — on *any* block, including misrouted ones — while paying the cheap tier's price wherever the guarantee permits it. The deployed router (§5) escalates on axis alignment rather than on DCI; its fidelity is measured directly in §6.3–§6.4.

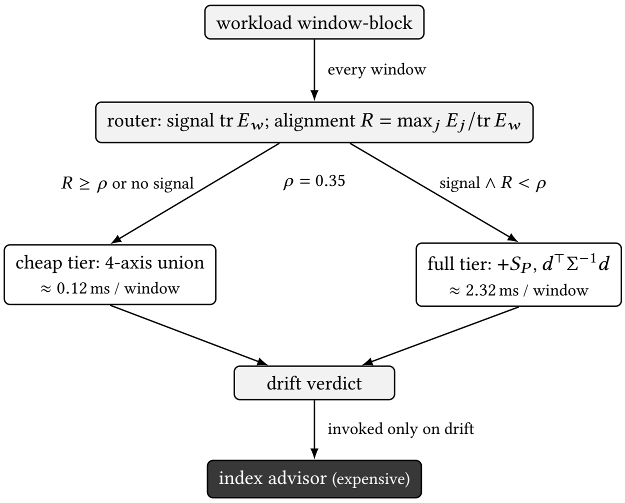

Figure 1: The routed monitoring layer. The four cheap axes (94% cheaper than the full feature set) are extracted *every* window and feed both tiers' router — two scalars, no eigendecomposition: a whitened signal-presence gate $\mathrm{tr}\, E_w$ and the axis-alignment share R. While some axis carries the drift ($R \ge \rho$) the cheap tier — a Bonferroni union over the four axes — is the detector; when drift is present but no axis dominates, the router pays for the positional axis and the full Mahalanobis detector. Whichever tier is active runs on every window; only its drift verdict invokes the expensive index advisor. DCI itself is reported as the complexity diagnostic throughout.

Theorem 1 turns the empirical frontier into a guarantee for the deployed axis-restricted detector: across the nine cells the measured 1-D-over-full deflection ratio tracks p_1 and the detection-AUC gap widens exactly as p_1 falls (single-cause cells $p_1 \ge 0.83$, gap ≈ 0.00; mixed cells $p_1 \approx 0.62$, gap 0.15–0.20). Two honest boundaries. Detection AUC is a *saturating* function of deflection, so the AUC frontier sits *above* the energy domain (a cell can carry $p_1 = 0.82$ of the energy and still reach AUC 1). And the white-noise model matches the deployed detector, which does not whiten; the whitened refinement — the participation ratio of $\Sigma^{-1/2} C \Sigma^{-1/2}$, strictly more powerful where the steady noise is correlated — is measured in §6.8 and prices as the full kernel.

5 Mechanism: Detectors and the DCI Router

Two tiers, placed by the cost structure. Let $d(t) = f(t) - f_{\text{steady}}$ be a window's deviation from the steady-state mean feature. Measured per-axis extraction cost is extremely skewed: the positional kernel S_P accounts for 2.208 ms of the 2.323 ms feature set (94%; §6), while the four remaining axes together cost ≈0.12 ms. The economically meaningful boundary is therefore not "one axis vs. five" but *cheap subspace vs. positional resolution.* The *cheap tier* extracts only the four cheap axes and fires iff any standardised coordinate exceeds the finite-sample threshold, $\max_j (d_j/\hat{\sigma}_j)^2 > F_{1,m-1}$ at level $\alpha/4$ (a Bonferroni union). The *full tier* additionally extracts S_P and scores the Mahalanobis distance $d^\top \Sigma^{-1} d$ against $F_{D,m-D}$ under the steady-window noise covariance Σ; since the axes are correlated (§3), this whitening is the statistically optimal linear fusion — under a Gaussian steady-state model it is the Neyman–Pearson statistic [26]. Both tiers read the workload's *position* relative to steady state, not its motion, so each is single-edged on a transient drift dip. Two designs we evaluated and rejected sit either side of this one: a whitened matched filter as the "cheap" arm is the full tier in disguise — it needs all five features and the eigenstructure (§6) — and a single fixed coordinate is strictly dominated by the union at essentially the same cost (§6). The cost structure, not the dimension count, dictates tier placement.

The router. Routing must separate drift *presence* from axis *alignment*, and must not mistake noise for diffuse drift. It maintains the running outer-product sum of standardised cheap deviations and two derived scalars — no eigendecomposition, no per-window matrix work. The *presence* statistic is the whitened excess energy $\operatorname{tr} E_w = \sum_t \|W_4\, d_{\text{cheap}}(t)\|^2 - 4n$, which sees drift in *any* cheap-subspace direction, including low-variance contrast directions of a correlated noise covariance that per-axis tests cannot see; a δ-rule gate ($\operatorname{tr} E_w > k_\sigma \sqrt{8n}$) keeps steady noise from registering as signal. The *alignment* statistic is the share of that energy captured by the best raw axis, $R = \max_j E_j / \operatorname{tr} E_w$ — the operational surrogate of the R_{axis} quantity in Theorem 1's bounds, matched to the max-statistic the cheap tier actually is. While $R \geq \rho$ (a dominant axis carries the drift; we fix $\rho = 0.35$ a priori) the union is the detector; when drift is present and $R < \rho$ — spread across axes, or carried by a correlated direction no single coordinate sees — the router extracts S_P *lazily inside the decision path* and escalates to the full tier. An optional audit cadence forces the full tier every K-th window, bounding exposure to drift visible only to the positional axis. DCI itself is not the routing threshold: drift spread over two *strong* axes has DCI ≈ 2 yet is exactly what a max test handles. DCI remains the reported complexity diagnostic, and the regime map (§6) remains its coordinate system.

Why not the composite. The scalar HSM is a fixed weighted average: a drift confined to one axis is attenuated by that axis's weight, while the noise on HSM aggregates all five. The composite is therefore a gating score, not a detector; §6 shows its detection AUC collapsing on a workload where a single axis is perfect.

The sufficiency tolerance. Calling the 1-D detector "sufficient" needs a tolerance: how large a multi-D-over-1-D AUC gap is still within noise? We derive it rather than set it by hand. DeLong's nonparametric estimator [10] gives the variance of the (correlated) AUC gap; pooled across seeds and evaluated at a fixed reference sample size N_{ref}, it yields a per-axis tolerance δ_d computed from each workload's own measured variance and stable as the experiment's seed count grows. An axis is sufficient iff its mean gap does not exceed δ_d.

6 Evaluation

We ask four questions. **RQ1**: is workload-drift complexity cheaply and reliably measurable? **RQ2**: does it predict which detector a workload needs, and where is the frontier? **RQ3**: does the geometry-routed two-tier layer preserve detection quality at materially lower monitoring cost? **RQ4**: does the routed layer drive a live index advisor end-to-end?

6.1 Setup

Three SQL workloads exercise the kernel: TPC-H (18 of its 22 query templates — those in the vendored generator's deterministic parameter pool; Q15, Q16, Q19, and Q22 are absent from it) and the Join Order Benchmark (JOB, 33 template families) are analytic; pgbench (the TPC-B-like five-statement script) is OLTP. All three are presented to the HSM kernel as SQL text. Drift trajectories come from a generator anchored on the phase profiles of a real MongoDB workload trace, under three configurations: *template* drift moves the template-structure axes, *volume* drift moves S_V, and *mixed* drift rotates the two so the pooled drift spans both axis groups. An arrival-only configuration is deliberately absent: the kernel's arrival axis S_P does not respond to arrival-timing changes while templates are held fixed—a measured property of this feature set, reported rather than hidden—so arrival drift cannot be isolated as a single-cause cell (§7). Each (workload, configuration) cell is run over 50 seeds; seeds are derived deterministically, so a re-run on a second machine reproduced every DCI and AUC to the printed digit. Parameters, stated once: each offline trajectory is N=41 scored windows (seven phases of six raw windows; features score adjacent windows); C is the sample covariance of the per-window feature vectors over the trajectory (mean-centred, ddof=1); the DeLong tolerance (§5) is evaluated at N_{ref}=15, yielding per-axis δ_d of 0.005–0.049 on the mixed cells; live blocks are 24 windows (§6.10). Code and data reproducing every number, figure, and table are available at https://github.com/stevezhai139/dci-reproducibility. Per-window detector cost is the wall-clock of the real kernel over five timing repeats, measured on an Apple M4 MacBook Pro (24 GB unified memory, macOS 26) single-threaded under CPython 3.13.3 with NumPy, SciPy, pandas, and the `fastdtw` and `pywavelets` libraries. The kernel's wavelet and DTW path depends on the last two libraries; only the absolute costs, not DCI or the regime map, depend on this platform.

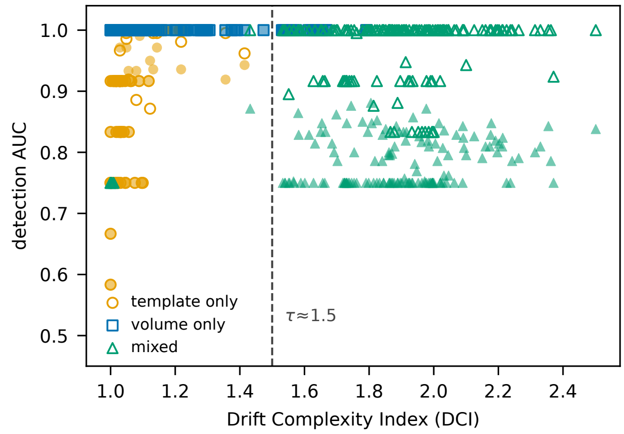

Figure 2: Detection AUC vs DCI for all 450 runs (50 seeds × 3 configurations × 3 workloads). Marker shape and colour denote the drift configuration; open markers are the multi-D detector, filled markers the 1-D detector — no information is colour-alone. At low DCI the 1-D detector ties the multi-dimensional detector; at high DCI (*mixed*) the 1-D detector falls while the multi-dimensional detector holds. Dashed line: the routing threshold $\tau \approx 1.5$.

6.2 The regime map (RQ1, RQ2)

Figure 2 plots detection AUC against DCI for all 450 runs. The structure is clean. Ordinary drift is *low-complexity*: template and volume cells sit at DCI $\in [1.0, 1.4]$, and there the 1-D detector ties the multi-dimensional detector. Heterogeneous (*mixed*) drift is *high-complexity*: DCI $\approx$ 1.8–2.0, and there the best single axis

Table 2: Per-cell regime map: the DCI and the 1-D-sufficient axes — those within the derived tolerance δ_d of the multi-dimensional detector. Every *mixed* cell has none; every *template/volume* cell has at least one. The first three workloads are relational (SQL); the MongoDB block (document store; identical drift schedule, 50 seeds per cell) reproduces the same structure — for it we list the verified dominant sufficient axis.

Workload	Drift	DCI	1-D-sufficient axes
TPC-H	template	1.02	S_R, S_T, S_A
TPC-H	volume	1.41	S_V, S_P
TPC-H	mixed	1.97	— (multi-D needed)
JOB	template	1.11	S_A
JOB	volume	1.13	S_V, S_P
JOB	mixed	1.92	— (multi-D needed)
pgbench	template	1.03	S_R, S_T
pgbench	volume	1.01	S_V, S_P
pgbench	mixed	1.75	— (multi-D needed)
MongoDB	template	1.04	S_R
MongoDB	volume	1.01	S_V
MongoDB	mixed	1.80	— (multi-D needed)

falls to AUC 0.75–0.82 while the multi-dimensional detector holds 0.91–1.00.

Table 2 gives the per-cell verdict under the derived tolerance δ_d. Every *mixed* cell has *no* 1-D-sufficient axis — the multi-dimensional detector is needed — and every template/volume cell has at least one. The two regimes are separated by an empty DCI band — the open interval $(1.41, 1.75)$ between the largest low-complexity cell mean and the smallest high-complexity cell mean — so a single threshold $\tau \approx 1.5$ separates them; across the 50 seeds per cell this separation holds at 95% confidence—every low-complexity cell's DCI 95% upper bound is ≤ 1.48 and every high-complexity cell's lower bound ≥ 1.68. The OLTP workload pgbench corroborates the analytic ones: its drift takes the same shape — low-DCI for template and volume, high-DCI for mixed. The bands sit where the trichotomy of Corollary 2 puts them: with the frontier $\tau = 1.5 = (1-\delta)^{-1}$, i.e. $\delta = \frac{1}{3}$, every low-complexity cell lies in the guaranteed-sufficiency zone and every mixed cell in the indeterminate zone $(1.5, 2.25]$, where the bounds alone do not decide and the experiment does — against a single axis, as the near-flat measured spectra ($p_1 \approx 0.63$–0.68, Table 5) predict.

Cross-data-model detection: the map transfers to MongoDB.. The construction is not tied to SQL. Applying the identical drift schedules to a MongoDB workload of 24 aggregation-pipeline templates (the kernel reads each query's pre-execution feature vector, so no database is executed) reproduces the same structure (Table 2, MongoDB block): single-axis drift stays low-complexity (DCI = 1.04, 1.01; one axis and the full kernel both at AUC $\geq$ 0.998), while mixed drift is high-complexity (DCI = 1.80; best single axis AUC 0.82 vs. multi-D 1.00)—the same collapse and recovery, in the same DCI bands, routed by the same τ. This is cross-*data-model* evidence at the detection layer; heterogeneous workload *types* (semi-structured, vector) remain future work.

A real workload sits in the regime. On a real SkyServer query log (99,969 queries, natural drift only) the same $O(D^2)$ probe reads a per-block DCI averaging 1.85 over 100 blocks and ranging over $[1.22, 2.41]$: real drift is predominantly high-complexity (90% of blocks above the frontier), with a 10% minority of low-complexity blocks. The heterogeneous regime where the multi-dimensional detector is required is thus not a synthetic artifact but the common case on this trace — and the honest reading cuts both ways. Selecting resolution by the frontier prices the trace at 2.12 ms per window, 90% of the always-multi-D monitoring cost: on a workload like this the probe's value is not a large saving but *foreknowledge* — it tells the operator, before any detector runs, that this workload must pay for full resolution, and exactly which tenth of it need not. (The trace carries no ground-truth onsets, so detection quality is not scored here; costs use the per-window timings of §6.)

Table 3: Policy bench on identical streams (9 cells $\times$ 50 seeds; pooled per configuration). Recall is strict exact-onset; AUC is graded ($-\log_{10} p$ scores); feature cost is the modeled per-axis extraction share of always-full. every-k policies under a +3-window lag tolerance reach only 0.48/0.64/0.61 (k=2) and 0.23/0.43/0.42 (k=4) on tmpl/vol/mixed; ADWIN is shown at both sensitivities.

policy	tmpl	vol	mixed	AUC_{mix}	feat. %
always full (5-D)	0.89	1.00	0.93	0.902	100
cheap union (Bonf.)	0.88	1.00	0.93	0.873	5.0
routed (ours)	0.88	1.00	0.93	0.873	5.0–6.4
routed + audit-8	0.88	1.00	0.93	0.882	17–18
best axis (oracle)	0.88	1.00	0.67	0.786	0.6–2.9
whitened filter (v1)	0.89	1.00	0.90	0.882	100
single axis, probed (v2)	0.77	0.22	0.50	0.662	21–22
every 2nd window	0.45	0.50	0.46	—	50
every 4th window	0.21	0.24	0.23	—	25
ADWIN (δ=.002/.3)	0.00	0.00	0.00	—	100
MCUSUM (Mahalanobis)	0.86	1.00	0.92	0.900	100
random 1-D sketch	0.88	1.00	0.92	0.877	100

6.3 Tier placement and the policy bench (RQ3)

Measured per-axis extraction cost is the design's anchor: S_P, the positional kernel, costs 2.208 ms of the 2.323 ms feature set (94%); the four cheap axes together cost ≈0.12 ms (5%). We therefore benchmark *policies* — who computes which features, when — on identical feature streams: the nine regime-map cells, 50 seeds each, strict exact-onset scoring, feature cost charged per axis actually computed (Table 3).

Five findings. **(i)** On binary firing the cheap union is statistically indistinguishable from the full detector: per-run recall is identical in 444 of the 450 paired runs, the six discordant runs split 3–3 (sign test p=1.0), and pooled over all 2,700 onsets both catch 2,535 — at 5% of the feature cost. The routed system inherits this; the router escalates only on the mixed cells (6.4% total) and never on drift-free streams. The full detector's remaining edge is *graded*: +0.03 AUC on mixed cells only. **(ii)** A single fixed axis — even chosen post hoc per run — collapses exactly where Theorem 1 says it must: mixed-cell recall 0.67, AUC 0.786. Two earlier designs bracket the lesson: the v1 gate's "cheap" arm (a whitened matched filter) matches the full detector because it *is* the full tier in disguise, at 100% feature cost (§6.8); a probed single-coordinate arm (v2) is strictly dominated by the union. Tier placement follows the cost structure, not the dimension count. **(iii)** Fixed-cadence auditing — the validation pattern of production auto-indexing services [6, 9] — catches transient drift with probability ≈$1/k$: recall 0.46/0.28/0.23 for k=2/3/4 on mixed cells, and lag tolerance does not rescue single-window onsets. **(iv)** ADWIN, designed for sustained mean shifts, never fires on single-window positional dips at either sensitivity — a structural

Table 4: Adversarial geometry (strict recall, 50 seeds). The router escalates exactly where per-axis tests are structurally blind (esc. = fraction of windows on the full tier). Drift visible only to S_P is the stated limitation: the audit cadence bounds it at $\approx 1/K$ for transient onsets.

geometry	Δ	union	routed (esc. %)	full	audit-8
axis-aligned	2.5	0.50	0.50 (0)	0.39	0.50
	3.5	0.84	0.84 (2)	0.75	0.83
diffuse (4 axes)	2.5	0.30	0.31 (4)	0.39	0.33
	3.5	0.59	0.61 (14)	0.72	0.61
correlated contrast	2.5	0.28	**0.96** (82)	0.99	0.97
	3.5	0.57	**1.00** (88)	1.00	1.00
S_P only	2.5	0.06	0.06 (0)	0.42	0.11
	3.5	0.05	0.06 (2)	0.75	0.15

mismatch, not a tuning failure, and the reason position-based windows are the right monitoring primitive here. **(v)** A multivariate CUSUM [14] on the same Mahalanobis stream ties the F-test at 100% feature cost: the choice of test statistic is not the story; *which features you pay for* is.

The saving at scale. These are per-window costs and one tier runs on *every* window, so at R windows/s the layer consumes $R\,c$ of CPU. At a reference R=100 windows/s the always-full layer occupies 23.5% of a core; the routed layer ≈1.2% when no escalation is active. Live, where fixed per-window overheads shared by both tiers compress the modeled gap, §6.10 measures 13–18% of always-full, end to end. Where windows are infrequent the choice is moot; that remains the honest boundary of the result.

6.4 Where the union fails: adversarial geometry

The nine workload cells are axis-aligned: some coordinate always carries the drift. The escalation path exists for the geometry where no coordinate does. We construct it directly in feature space (steady $\mathcal{N}(0, \Sigma_0)$, transient mean shifts of total size Δ, 50 seeds): drift along a *contrast direction* of an equicorrelated noise covariance (r=0.8) puts $\Delta/2$ on each axis — inside per-axis noise — while its whitened norm is 2.24Δ (Table 4).

Three boundaries, quantified. On axis-aligned drift the cheap tier is not merely adequate — it *beats* the full detector at low magnitudes (0.50 vs 0.39 at Δ=2.5: the five-degree-of-freedom threshold taxes the full statistic). On the correlated contrast the union is structurally blind (0.28) and the router recovers the full tier's power (0.96 at 82% escalation): it pays for positional resolution exactly while the geometry demands it, and returns to 5% elsewhere. Diffuse equal-share drift shows the classic max-vs-sum gap (≤0.13 recall) — real but modest at D=4. Drift visible *only* to S_P is invisible to any cheap-subspace statistic: the audit cadence bounds transient exposure at $\approx 1/K$ and persistent exposure at delay $\leq K$; this is the design's stated limitation, priced rather than hidden.

6.5 Threshold sensitivity

The routing frontier $\tau \approx 1.5$ is not a tuned constant. Figure 3 sweeps it over all 450 runs. As τ falls toward 1 every window routes to the multi-dimensional detector — full accuracy at full cost; as τ rises past the high-complexity cells the mixed-drift gain is lost. Between the two lies a plateau: for $\tau \in [1.2, 1.5]$ routing by DCI retains ≥ 99% of the multi-D-over-1-D gain while paying only 37–48% of the multi-D cost, and the retained gain collapses only once τ crosses 1.75 and the *mixed* cells are misrouted (63.9%

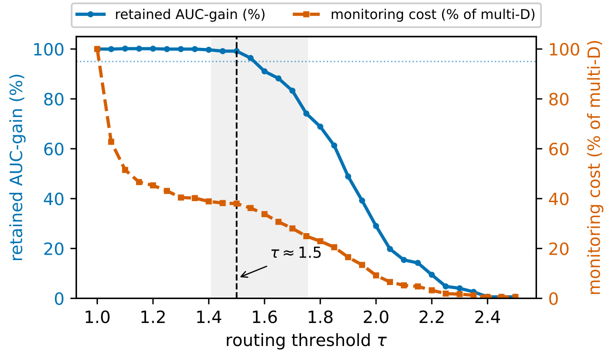

Figure 3: Threshold sensitivity. Retained multi-D-over-1-D AUC-gain (left axis) and monitoring cost as a fraction of the multi-dimensional detector (right axis) versus the routing threshold τ, over all 450 runs. The shaded band is the empty per-cell DCI interval (1.41, 1.75)**;** $\tau \approx 1.5$ **sits at the knee where routing by the frontier keeps** ≥ 99% **of the AUC gain at** ≈ 38% **of the modeled cost.**

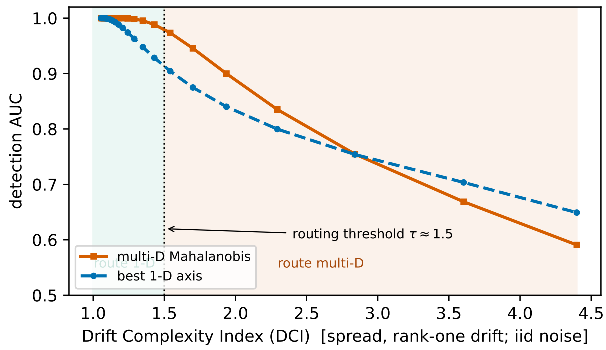

Figure 4: Why $\tau \approx 1.5$ is principled: detection AUC of the multi-dimensional detector and the best single axis versus DCI, in a synthetic model matched to the two detectors (rank-one drift along the hardest, maximally spread direction; iid steady noise). The curves separate at DCI ≈ 1.5**: the frontier sits at the onset of the detectability gap.**

retained at τ=1.75, 21.1% at τ=2.0). At the cell level no cell's mean DCI lies inside the open band (1.41, 1.75) — its endpoints are themselves the two nearest cell means — so any τ within it routes all nine cells identically. The frontier is therefore robust — $\tau \approx 1.5$ sits at the efficient knee, the largest (cheapest) threshold that still captures essentially all the gain. This is a statement about the *diagnostic* frontier — which resolution a cell needs — and it is what licenses the regime map's binary reading. The deployed router (§5) thresholds axis alignment rather than DCI, and is similarly insensitive to its own knob: raising ρ from 0.35 to 0.50 leaves every recall in Table 3 unchanged, at higher escalation cost (5.0% ⟶ 5–17% on single-cause cells, 6.4 ⟶ 27% on mixed) — ρ trades cost, not fidelity, in this range.

Why the frontier is principled. The location of τ is not an accident of these workloads. In a synthetic model matched to the two detectors—drift of graded magnitude along the *hardest*, maximally spread direction, under iid and correlated steady noise—the multi-D-over-1-D detectability gap tracks DCI: at most 0.025 AUC below DCI ≈ 1.25, climbing steeply across the empty band (0.04–0.06 over DCI 1.3–1.45), and largest (≈ 0.07) over DCI ≈ 1.5–1.9, precisely the band the router escalates (Figure 4). The threshold thus sits on the steep rise of the detectability gap, just below its peak—model evidence that $\tau \approx 1.5$ marks where multi-dimensional fusion pays most, not a constant tuned to these cells. (At the sweep's far end, where ever-weaker drift inflates DCI toward the noise floor, both detectors degrade together: that region is noise-dominated, unlike the strong heterogeneous drift of the mixed cells, whose multi-dimensional AUC stays 0.91–1.00.) The same sweep probed the converse risk, a low-DCI drift hidden *off-axis*, and found none: a drift with DCI low enough to route to the 1-D detector must dominate the steady noise, and a drift that strong remains detectable on a single axis even when maximally spread. Low DCI genuinely implies 1-D detectability in this model; the frontier's residual dependence on the steady-noise structure is discussed in §8.

6.6 The tolerance, and further checks

Three checks support the result. (i) The composite is not a detector: the scalar HSM's detection AUC falls to 0.68 where a single axis is perfect, confirming a weighted average dilutes single-axis drift. (ii) The tolerance δ_d is verified, not assumed: against known-truth synthetic data its DeLong standard error matches a stratified bootstrap to within 6% and its 95% interval covers at 0.94–0.96 (inverse-variance pooling, biased for AUC, covers only 51%). (iii) The regime map is recovered from as few as 10 seeds, so 50 is ample.

6.7 Ablation: why the participation ratio?

DCI is one of several ways to summarise a spectrum's concentration; does the routing verdict depend on the choice? Table 5 recomputes, per cell, three alternatives: the *effective rank* $\exp(-\sum_i p_i \ln p_i)$ (the Hill number of order one [15]); the *stable rank* $\mathrm{tr}(C)/\lambda_{\max}$; and the dominant-mode share $p_1 = \lambda_{\max}/\mathrm{tr}(C)$. Every one separates the low-complexity (template/volume) cells from the high (mixed) cells with an empty band (for DCI, on (1.41, 1.75); likewise for the others), so the verdict is robust to the choice of functional. The participation ratio is nonetheless the right runtime statistic: it is the *only* one of the four computable without an eigendecomposition — a trace and a Frobenius norm ($O(D^2)$; §4), whereas effective rank needs the whole spectrum and stable rank and p_1 need the top eigenvalue ($O(D^3)$ on every routed block). Same verdict, lowest cost.

6.8 The whitened refinement

DCI reads the raw drift covariance; the steady noise Σ is left to the detectors. The noise-aware refinement — DCI_w, the participation ratio of $\Sigma^{-1/2} C\, \Sigma^{-1/2}$ — describes the drift a *whitened* detector family sees. On the identical trajectories and seeds as the regime map (the raw column reproduces Table 2 to the printed digit), whitening only *concentrates* the drift: $\mathrm{DCI}_w \le \mathrm{DCI}$ on all nine cells, collapsing the volume cells to ≈1.01 and JOB-mixed from 1.92 to 1.08, while pgbench-mixed is unchanged at 1.75 — its Σ is near-isotropic, so whitening is a no-op there. The statistic is stable over the covariance ridge in $[10^{-8}, 10^{-6}]$; larger ridges interpolate back toward the raw value, as un-whitening should.

The detector this concentrated spectrum promises exists: the whitened *matched filter* — the squared projection of the whitened deviation on the dominant whitened drift mode, the strictly-more-powerful 1-D detector the white-noise model of Theorem 1 anticipates. Under the map's own tolerance rule, it is sufficient in *every* cell — including the three mixed cells, where the best raw axis loses 0.16–0.21 AUC yet the whitened filter comes within 0.015, 0.001, and 0.035 of the multi-dimensional detector (JOB, TPC-H, and pgbench; the last with the loosest floor, on the noisiest cell). This does not create a third routing option, because the whitened filter is the full-kernel tier in disguise: $z = \Sigma^{-1/2}(f - \mu)$ needs all five axes of f on every window — the 2.32 ms feature cost — whereas the cheap tier's ≈0.12 ms comes precisely from *not* computing S_P. The monitoring layer thus has two cost tiers, not three: axis-restricted (cheap) and full-kernel (expensive, in either its Mahalanobis or matched-filter form). Raw DCI routes *between* the tiers — erring only toward escalation, since $\mathrm{DCI}_w \le \mathrm{DCI}$ — while DCI_w describes structure *within* the full tier. Table 2 is therefore the map for the detector family a cheap tier can actually deploy.

6.9 The regime is not tied to the kernel

DCI reads only the covariance of a per-window feature vector (§4), so its regime should not depend on the particular similarity axes it is fed. We test this directly: on the *same* drift windows and seeds, we recompute DCI and the 1-D/multi-D detection AUC under three representations that use *none* of the HSM kernel's machinery (Table 6)—a generic three-axis similarity (adjacent-window template cosine, query-count ratio, table Jaccard), a raw per-window template-frequency vector, and a table-incidence bag.

Two findings. First, the generic three-axis similarity—which shares no construction with the HSM-style feature—reproduces the regime almost exactly (single-cause drift low-complexity, mixed high-complexity, and the same frontier $\tau \approx 1.5$): the low-complexity of ordinary drift is a property of the drift, not of that kernel. Second, the routing rule is valid in *every* representation. Wherever DCI is low the 1-D detector matches the multi-dimensional one; wherever DCI is high the 1-D detector falls and the multi-dimensional detector recovers it—including the raw representations, where even ordinary *template* drift turns high-complexity (DCI up to 4.0) because it moves many raw dimensions, and DCI correctly routes it to the multi-dimensional detector. What changes across representations is therefore not the router's validity but the *operating point*: a bounded-similarity featurization places ordinary drift at low complexity, so the cheap detector usually suffices; a raw high-dimensional one does not. DCI predicts detector sufficiency on whatever features the monitoring layer computes.

6.10 End-to-end deployment (RQ4)

To confirm the routed detector drives a real adaptation loop, we connect it to live, *off-the-shelf* index advisors — Dexter 0.6.3, a HypoPG-based recommender (HypoPG 1.4.2), on PostgreSQL 16.9 (TPC-H, SF 1), and an equality–sort–range recommender on MongoDB 8.0.8 (a 5M-document store) — over 10 randomized complete-block replicates of a mixed-drift schedule whose seeded onsets are ground truth: six phases of four windows per 24-window block, onsets alternating *template* moves (half-swap of the active template set on PostgreSQL; a switch of the dominant document modality on MongoDB) and *volume* moves (20↔32 relational, 20↔48 document queries per window). Offline pre-validation places these blocks in the heterogeneous regime (pooled

Table 5: Ablation: the participation ratio (DCI) against three alternative spectrum-concentration measures, per cell (mean over 50 seeds, all three workloads). Every measure separates the LOW (template/volume) cells from the HIGH (mixed) cells; only DCI needs no eigendecomposition. All functionals are computed per seed and then averaged, so the tabulated stable rank need not equal the reciprocal of the tabulated p_1.

Workload	Drift	DCI	eff. rank	stable rank	p_1
TPC-H	template	1.02	1.07	1.01	0.99
TPC-H	volume	1.41	1.57	1.22	0.83
TPC-H	mixed	1.97	2.17	1.62	0.63
JOB	template	1.11	1.22	1.05	0.95
JOB	volume	1.13	1.26	1.06	0.94
JOB	mixed	1.92	2.11	1.62	0.63
pgbench	template	1.03	1.08	1.02	0.98
pgbench	volume	1.01	1.02	1.00	1.00
pgbench	mixed	1.75	1.85	1.52	0.68

Table 6: DCI and 1-D/multi-D detection AUC under three *kernel-free* feature representations (pooled over the three SQL workloads, 50 seeds; each cell reads "DCI 1D/mD"). A bounded-similarity representation reproduces the regime of §6.2; raw representations move ordinary drift to high complexity—but in all of them high DCI coincides with 1-D insufficiency, so the DCI router stays valid.

representation	template	volume	mixed
generic 3-axis sim.	1.11 .93/.93	1.04 1.0/1.0	1.89 .79/.97
raw template-freq.	4.02 .78/.95	1.87 .97/1.0	3.18 .82/.97
table-incidence bag	2.25 .80/.86	1.03 .97/.97	1.81 .87/.91

onset DCI 2.04 PostgreSQL, 1.72 MongoDB). The advisor is a black-box downstream consumer: the monitoring layer decides only *when* to invoke it, so we score the layer's own firings against ground truth — recall, false alarms — and its *measured* per-window detector cost, not the advisor's economic value (§7).

A single direction is not enough. First, the negative result the theory predicts. A fixed single-direction detector — the whitened matched filter that an earlier revision of this system used as its "cheap" arm (§6.3) — run always-on over the same schedule locks onto the direction that comes to dominate its trajectory (here the volume axis) and from that point misses every later template onset: recall 0.60 on both engines, with the same deterministic signature (hits at windows 5, 9, 17; misses at 13, 21; in 10/10 blocks) on a relational and a document store — an outcome consistent with the insufficiency regime of Corollary 2, at the decision level. Whether a miss costs query latency depends on who owns the indexes. On PostgreSQL the structural backbone covers the mixed queries' access paths and the damage is nil — an honest null. On MongoDB the advisor owns each modality's index: after the missed template onsets the document phase runs unindexed at 10.9× the reference latency until the next volume onset — one the detector *can* see — repairs the indexes as a side effect; the pre-registered +2-window contrast summarizes this conservatively at 1.23.

The routed system, measured. Table 7 reports the three-arm study — always-full, the routed system, and the always-cheap union — on identical workload realizations per block (fingerprint-checked). The routed detector catches *every* onset of both types on both engines (T 30/30, V 20/20), with *half* the full detector's false alarms on PostgreSQL (0.10 vs 0.20 per block) and a third on MongoDB (0.30 vs 0.80); the arms disagree only on windows where always-full false-alarms (decision agreement 99.2%/95.8%). The measured conditional detector path — the four cheap axes

Table 7: Live mixed drift, three arms in front of the same advisors: 10 RCB blocks per engine, onsets alternating template (T, 30 total) and volume (V, 20 total), identical workload realizations across arms (fingerprint-checked). FA = fires per block outside an onset or its +1 window; det = *measured* per-window detector-path cost (feature extraction + decision); agree = share of windows whose firing decision equals always-full's.

Engine	Arm	T	V	FA	det (ms)	vs full	agree
PostgreSQL	always-full	30	20	0.20	11.14	100%	—
PostgreSQL	**routed**	30	20	**0.10**	**2.04**	18.3%	99.2%
PostgreSQL	always-cheap	30	20	0.10	2.10	18.8%	99.2%
MongoDB	always-full	30	20	0.80	2.50	100%	—
MongoDB	**routed**	30	20	**0.30**	**0.33**	13.1%	95.8%
MongoDB	always-cheap	30	20	0.30	0.33	13.1%	95.8%

every window, S_P extracted lazily inside the decision path on escalation — costs 2.04 ms per window against always-full's 11.14 ms on PostgreSQL (18.3%) and 0.33 vs 2.50 ms on MongoDB (13.1%): 5.5–7.6× cheaper, measured end to end rather than modeled from per-axis constants. Paired post-onset/steady latency contrasts are 0.99–1.01 on both engines: the monitoring layer does not perturb the workload it watches. The router reports zero escalations across all 240 windows per engine — live drift here is axis-aligned, exactly the regime the map assigns it — so the routed and always-cheap arms coincide on this schedule; the geometry where they part is quantified in §6.4, and the audit cadence prices the S_P-only blind spot.

7 Discussion and Future Work

What the result says. The empirical core — ordinary workload drift is *low-complexity*, occupying essentially a single mode, while only heterogeneous drift spreads across several — is a statement about how workloads move, not merely about detectors: it is why a nearly free always-on layer suffices almost always, and the bounds of §4 say when it will not. Reading only the covariance of a per-window feature vector, DCI is indifferent to *how* that vector is produced and *what* its verdict triggers — a general monitoring-resolution knob for self-driving data systems, of which index-advisor gating is one instance.

The domain boundary, drawn by the primitive itself. The scope of the cheap path is not asserted; it is measured and proved. *Covered:* abrupt-onset, single-cause drift—the low-DCI zone where Corollary 1 guarantees the 1-D detector retains a $1 - \delta$ share of the full power—demonstrated across three relational workloads and a document store, and live on two engines (§6.10). *Covered by escalation:* heterogeneous drift. Corollary 2 proves no single direction suffices once DCI $> (1-\delta)^{-2}$; the router escalates on exactly the geometry this describes — drift no single axis carries — measured directly in §6.4 (union 0.28, routed 0.96 on the correlated contrast), and live the routed layer holds recall 1.00 across the boundary against the fixed single-direction detector's 0.60 (§6.10). What this paper does not develop is that path's end-to-end economics. *Outside the domain:* gradual (non-abrupt) drift, for which the position detectors were not designed (§5); arrival-only drift, which this kernel's S_P cannot isolate (§6); and drift visible *only* to the positional axis, which no cheap-subspace statistic can see between audits — priced at $\approx 1/K$ in §6.4. A system built on the layer inherits this boundary—and can read it, in advance, off the same number.

Future work. Four directions follow. **(i) Economics of the multi-dimensional path.** §6.10 validates the routed advisor live in both regimes: the cheap detector where Corollary 1 covers it, escalation where Corollary 2 proves a fixed cheap detector cannot go. What remains open is not *whether* the multi-dimensional path is needed, nor whether the gate reaches it (it does, decision-matched to always-multi-D), but its end-to-end *economics* — plan quality, invocations avoided, and when paying its cost is worthwhile — the journal-scale sequel. **(ii) Representations.** Re-derive the regime map under alternative per-window features (a raw query-frequency vector, or learned query/plan embeddings) to test how far the frontier $\tau \approx 1.5$ travels and to place DCI on modern learned workload models. **(iii) Engines and data models.** The regime map already transfers to a document store (§6.2); extending it to column and vector stores, and to heterogeneous workload *types* beyond the controlled drift model, is the natural next step. **(iv) Adaptivity and composition.** Let the frontier τ and the router's alignment threshold ρ self-tune online as a workload's drift geometry shifts, and compose DCI's *select-on-complexity* routing with the *escalate-on-event* hierarchical testing of §2 [44, 45] into a single two-layer monitor.

8 Threats to Validity

Scope: a compact end-to-end. The live deployment (§6.10) is a 10-block-per-regime validation on two engines — detection recall, advisor invocations, and query latency — not a full economic study of plan quality and invocations avoided across workloads; that study is future work (§7). Because the three detector configurations run sequentially, raw cross-configuration latency ratios include machine-state drift; the mixed-regime deployment therefore reads latency through the within-configuration post-onset/steady contrast, whose negative control (the gated leg, which builds the reference's indexes) comes out at 1.01.

Online deployment. The end-to-end validation runs the gate over controlled blocks with a fixed threshold τ against a seeded ground-truth schedule. A fully *online* deployment—the gate running continuously on a production stream, τ self-tuning as the workload's drift geometry shifts, and the gate robust to drift in its own steady-state statistics—is future work (§7), as is the advisor's economic payoff under live gating.

Construct: synthetic drift. Drift trajectories come from a generator anchored on the phase profiles of a real workload trace; they span template, volume, and mixed drift but are not an in-the-wild deployment. We corroborate the analytic workloads with an OLTP one (pgbench) and a full document-store regime map on MongoDB (§6.2), but a field study remains future work. The model is also abrupt-onset by construction—phase transitions produce the transient dips the position detectors are built for (§5); gradual or continuous drift, where motion-based statistics become competitive, lies outside the validated domain (§7).

External: the frontier and the feature set. The regime map is demonstrated on one workload-similarity kernel. DCI depends only on the feature covariance (§3), so the construction transfers, but the frontier $\tau \approx 1.5$ is empirical across three SQL workloads and a document store, not a proven constant — a different feature set could place it elsewhere (§7). DCI also reads only the drift covariance, not the steady-state *noise* covariance Σ; the frontier is therefore Σ-dependent, and a noise-aware refinement — the participation ratio of the *whitened* drift covariance $\Sigma^{-1/2} C\, \Sigma^{-1/2}$ — is a natural next step. That refinement is measured in §6.8: $\mathrm{DCI}_w \le \mathrm{DCI}$ on every cell, so the deployed raw-DCI router errs only toward escalation, and the refinement's own matched filter prices as the full-kernel tier — the frontier's Σ-dependence bounds *which* cheap axis suffices, not whether the two-tier routing stands.

Internal: cost measurement. Absolute per-window costs are single-machine wall-clock on one Apple M4 platform and will shift with hardware and library versions; the per-axis cost skew (94% in S_P), the measured live cost ratios, and the DCI/regime-map results are platform-independent by construction, since DCI is dimensionless and the map is defined on AUC and DCI alone.

9 Conclusion

Workload drift has a measurable complexity — the effective number of independent modes its motion occupies — and the Drift Complexity Index estimates it in $O(D^2)$ with no eigendecomposition. We proved that DCI bounds detector sufficiency — the best 1-D detector's power is the DCI-bracketed dominant-mode share — and confirmed it empirically: ordinary drift is *near* rank-one, so the cheap tier suffices, while only heterogeneous drift needs full resolution. That regime is reproduced by a kernel-free similarity and exhibited by a real query-log trace, so it is a property of the drift, not of one kernel. Reading this off *before* any detector runs places the always-on monitoring layer at the cheapest sufficient tier: live on two engines it matches the full detector's recall with half its false alarms at 5.5–7.6× lower measured cost, and escalates to full resolution exactly where the drift geometry demands it. The practitioner rule is short: measure a workload's DCI, then deploy the detector its regime — and the bound — call for. DCI is a monitoring-resolution primitive for self-driving data systems; advisor gating is its first application.

10 Artifacts

All code, data, and run artifacts reproducing every number, figure, and table in this paper are publicly available at https://github.com/stevezhai139/dci-reproducibility. The repository contains the workload-similarity kernel and the DCI implementation, the drift generators and regime-map pipeline (§6), the kernel-free representations and the SkyServer adapter (§6.9), the DCI-routed gate and both live end-to-end harnesses with their ordinary and mixed drift schedules (§6.10), and the official run outputs with per-run provenance (seeds, environment lock, engine and driver versions). `REPRODUCE.md` maps each paper artifact to its script, its official run directory, and the expected output; a read-only preflight script verifies the live environment (PostgreSQL 16.9

with HypoPG and Dexter; MongoDB 8.0.8) before any database experiment. The offline pipeline is fully deterministic — every cell is keyed by a stable per-seed hash — and requires no database.

Acknowledgments

Portions of this paper's code and text were developed with the assistance of a generative AI system (Anthropic's Claude): the experiment-harness variants, run and analysis scripts, and initial drafts of parts of the text were AI-generated under the authors' direction and subsequently reviewed, edited, and validated by the authors. All experimental results were measured by the authors on the pinned environment of §6 from the released repository; references were selected and verified by the authors.

References

[1] Sanjay Agrawal, Surajit Chaudhuri, Lubor Kollár, Arnd Christian Marathe, Vivek R. Narasayya, and Manoj Syamala. 2004. Database Tuning Advisor for Microsoft SQL Server 2005. In *Proc. 30th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, Toronto, Canada, 1110–1121. doi:10.1016/B978-012088469-8.50097-8

[2] Shruti Arora, Rinkle Rani, and Nitin Saxena. 2024. A Systematic Review on Detection and Adaptation of Concept Drift in Streaming Data Using Machine Learning Techniques. *WIREs Data Mining and Knowledge Discovery* 14, 4 (2024), e1536. doi:10.1002/widm.1536

[3] Albert Bifet and Ricard Gavaldà. 2007. Learning from Time-Changing Data with Adaptive Windowing. In *Proc. SIAM International Conference on Data Mining (SDM)*. 443–448. doi:10.1137/1.9781611972771.42

[4] Matteo Brucato, Tarique Siddiqui, Wentao Wu, Vivek Narasayya, and Surajit Chaudhuri. 2024. Wred: Workload Reduction for Scalable Index Tuning. *Proc. ACM on Management of Data (SIGMOD)* 2, 1 (2024).

[5] Yang Cao, Andrew Thompson, Meng Wang, and Yao Xie. 2019. Sketching for Sequential Change-Point Detection. *EURASIP Journal on Advances in Signal Processing* 2019, 42 (2019). doi:10.1186/s13634-019-0635-3

[6] Sunil Chakkappen, Shreya Kunjibettu, Daniel McGreer, Masoomeh Javidi Kishi, Hong Su, Mohamed Ziauddin, Mohamed Zait, Zhan Li, and Yuying Zhang. 2025. Automatic Indexing in Oracle. *Proc. VLDB Endow.* 18, 12 (2025), 4924–4937. doi:10.14778/3750601.3750616

[7] Surajit Chaudhuri and Vivek R. Narasayya. 1997. An Efficient, Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proc. 23rd International Conference on Very Large Data Bases (VLDB)*. 146–155.

[8] Surajit Chaudhuri and Vivek R. Narasayya. 1998. AutoAdmin "What-If" Index Analysis Utility. In *Proc. ACM SIGMOD International Conference on Management of Data*. 367–378. doi:10.1145/276304.276337

[9] Sudipto Das, Miroslav Grbic, Igor Ilic, Isidora Jovandic, Andrija Jovanovic, Vivek R. Narasayya, Miodrag Radulovic, Maja Stikic, Gaoxiang Xu, and Surajit Chaudhuri. 2019. Automatically Indexing Millions of Databases in Microsoft Azure SQL Database. In *Proc. ACM SIGMOD*. 666–679. doi:10.1145/3299869.3314035

[10] Elizabeth R. DeLong, David M. DeLong, and Daniel L. Clarke-Pearson. 1988. Comparing the Areas under Two or More Correlated Receiver Operating Characteristic Curves: A Nonparametric Approach. *Biometrics* 44, 3 (1988), 837–845. doi:10.2307/2531595

[11] João Gama, Pedro Medas, Gladys Castillo, and Pedro Pereira Rodrigues. 2004. Learning with Drift Detection. In *Advances in Artificial Intelligence – SBIA 2004 (Lecture Notes in Artificial Intelligence, Vol. 3171)*. 286–295. doi:10.1007/978-3-540-28645-5_29

[12] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A Survey on Concept Drift Adaptation. *Comput. Surveys* 46, 4, Article 44 (2014). doi:10.1145/2523813

[13] Peiran Gao, Eric Trautmann, Byron Yu, Gopal Santhanam, Stephen Ryu, Krishna Shenoy, and Surya Ganguli. 2017. A Theory of Multineuronal Dimensionality, Dynamics and Measurement. doi:10.1101/214262 bioRxiv 214262.

[14] John D. Healy. 1987. A Note on Multivariate CUSUM Procedures. *Technometrics* 29, 4 (1987), 409–412. doi:10.1080/00401706.1987.10488268

[15] M. O. Hill. 1973. Diversity and Evenness: A Unifying Notation and Its Consequences. *Ecology* 54, 2 (1973), 427–432. doi:10.2307/1934352

[16] Marc Holze and Norbert Ritter. 2007. Towards Workload Shift Detection and Prediction for Autonomic Databases. In *Proc. ACM First Ph.D. Workshop in CIKM (PIKM)*. doi:10.1145/1316874.1316892

[17] Hanxian Huang, Tarique Siddiqui, Rana Alotaibi, Carlo Curino, Jyoti Leeka, Alekh Jindal, Jesús Camacho-Rodríguez, Yuanyuan Tian, and Yi Zhao. 2024. Sibyl: Forecasting Time-Evolving Query Workloads. *Proc. ACM on Management of Data* 2, 1, Article 41 (2024). doi:10.1145/3639308

[18] Tze Leung Lai. 1995. Sequential Changepoint Detection in Quality Control and Dynamical Systems. *Journal of the Royal Statistical Society: Series B* 57, 4 (1995), 613–658. doi:10.1111/j.2517-6161.1995.tb02052.x

[19] Tze Leung Lai. 1998. Information Bounds and Quick Detection of Parameter Changes in Stochastic Systems. *IEEE Transactions on Information Theory* 44, 7 (1998), 2917–2929. doi:10.1109/18.737522

[20] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proc. VLDB Endowment* 17, 8 (2024), 1939–1952.

[21] Patrick A. Lee and T. V. Ramakrishnan. 1985. Disordered Electronic Systems. *Reviews of Modern Physics* 57, 2 (1985), 287–337. doi:10.1103/RevModPhys.57.287

[22] Guanli Liu and Renata Borovica-Gajic. 2025. DriftBench: Defining and Generating Data and Query Workload Drift for Benchmarking. arXiv:2510.10858 [cs.DB]

[23] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, João Gama, and Guangquan Zhang. 2019. Learning under Concept Drift: A Review. *IEEE Transactions on Knowledge and Data Engineering* 31, 12 (2019), 2346–2363. doi:10.1109/TKDE.2018.2876857

[24] Lin Ma, Dana Van Aken, Ahmed Hefny, Gustavo Mezerhane, Andrew Pavlo, and Geoffrey J. Gordon. 2018. Query-based Workload Forecasting for Self-Driving Database Management Systems. In *Proc. ACM SIGMOD International Conference on Management of Data*. 631–645. doi:10.1145/3183713.3196908

[25] Michael J. Mior, Kenneth Salem, Ashraf Aboulnaga, and Rui Liu. 2016. NoSE: Schema Design for NoSQL Applications. In *Proc. IEEE 32nd International Conference on Data Engineering (ICDE)*. 181–192. doi:10.1109/ICDE.2016.7498240

[26] Jerzy Neyman and Egon S. Pearson. 1933. On the Problem of the Most Efficient Tests of Statistical Hypotheses. *Philosophical Transactions of the Royal Society A* 231 (1933), 289–337. doi:10.1098/rsta.1933.0009

[27] Rafiullah Omar, Justus Bogner, Joran Leest, Vincenzo Stoico, Patricia Lago, and Henry Muccini. 2024. How to Sustainably Monitor ML-Enabled Systems? Accuracy and Energy Efficiency Tradeoffs in Concept Drift Detection. In *Proc. 10th International Conference on ICT for Sustainability (ICT4S)*. doi:10.1109/ICT4S64576.2024.00010

[28] E. S. Page. 1954. Continuous Inspection Schemes. *Biometrika* 41, 1/2 (1954), 100–115. doi:10.1093/biomet/41.1-2.100

[29] Zhicheng Pan, Y. Zhang, C. Yang, Ahmad Ghazal, Rong Zhang, H. Hui, X. Wu, Y. Dong, and X. Zhou. 2025. Hyper: Hybrid Physical Design Advisor with Multi-agent Reinforcement Learning. In *Proc. IEEE 41st International Conference on Data Engineering (ICDE)*. 1565–1578.

[30] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tomasic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. In *Proc. 8th Biennial Conference on Innovative Data Systems Research (CIDR)*.

[31] R. Malinga Perera, Bastian Oetomo, Benjamin I. P. Rubinstein, and Renata Borovica-Gajic. 2021. DBA Bandits: Self-Driving Index Tuning under Ad-hoc, Analytical Workloads with Safety Guarantees. In *Proc. IEEE 37th International Conference on Data Engineering (ICDE)*. 600–611. doi:10.1109/ICDE51399.2021.00058

[32] R. Malinga Perera, Bastian Oetomo, Benjamin I. P. Rubinstein, and Renata Borovica-Gajic. 2023. No DBA? No Regret! Multi-Armed Bandits for Index Tuning of Analytical and HTAP Workloads with Provable Guarantees. *IEEE Transactions on Knowledge and Data Engineering* 35, 12 (2023), 12855–12872. doi:10.1109/TKDE.2023.3271664

[33] Alfréd Rényi. 1961. On Measures of Entropy and Information. In *Proc. 4th Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1. 547–561.

[34] A. Reungsilpkolkarn, A. Sangmanee, and C. Phiphatkul. 2026. Adaptive Indexing for Schema-Less Temporal Databases Using Hierarchical Similarity Measurement. In *Proc. 9th Int. Conf. on Information and Computer Technologies (ICICT)*. Honolulu, HI, USA, 60–65. doi:10.1145/3803291.3803321

[35] Olivier Roy and Martin Vetterli. 2007. The Effective Rank: A Measure of Effective Dimensionality. In *Proc. European Signal Processing Conference (EUSIPCO)*. 606–610.

[36] Franklin E. Satterthwaite. 1946. An Approximate Distribution of Estimates of Variance Components. *Biometrics Bulletin* 2, 6 (1946), 110–114. doi:10.2307/3002019

[37] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proc. ACM SIGMOD International Conference on Management of Data*. 1009–1024. doi:10.1145/3035918.3064029

[38] Junxiong Wang, Immanuel Trummer, and Debabrota Basu. 2021. UDO: Universal Database Optimization using Reinforcement Learning. *Proc. VLDB Endowment* 14, 13 (2021), 3402–3414. doi:10.14778/3484224.3484236

[39] Xiaoying Wang, Wentao Wu, Vivek R. Narasayya, and Surajit Chaudhuri. 2025. Esc: An Early-Stopping Checker for Budget-aware Index Tuning. *Proc. VLDB Endowment* 18, 5 (2025), 1278–1290.

[40] Zijia Wang, Haoran Liu, Chen Lin, Zhifeng Bao, Guoliang Li, and Tianzheng Wang. 2024. Leveraging Dynamic and Heterogeneous Workload Knowledge to Boost the Performance of Index Advisors. *Proc. VLDB Endowment* 17, 7 (2024), 1642–1654. doi:10.14778/3654621.3654631

[41] Geoffrey I. Webb, Roy Hyde, Hong Cao, Hai Long Nguyen, and François Petitjean. 2016. Characterizing Concept Drift. *Data Mining and Knowledge Discovery* 30, 4 (2016), 964–994. doi:10.1007/s10618-015-0448-4

[42] Bernard L. Welch. 1947. The Generalization of 'Student's' Problem when Several Different Population Variances are Involved. *Biometrika* 34, 1–2 (1947), 28–35. doi:10.1093/biomet/34.1-2.28

[43] Chenning Wu, Sifan Chen, Wentao Wu, Yinan Jing, Zhenying He, Kai Zhang, and X. Sean Wang. 2026. UTune: Towards Uncertainty-Aware Online Index Tuning. arXiv:2601.18199.

[44] Shujian Yu and Zubin Abraham. 2017. Concept Drift Detection with Hierarchical Hypothesis Testing. In *Proc. SIAM International Conference on Data Mining (SDM)*. 768–776. doi:10.1137/1.9781611974973.86

[45] Shujian Yu, Xiaoyang Wang, and José C. Príncipe. 2018. Request-and-Reverify: Hierarchical Hypothesis Testing for Concept Drift Detection with Expensive Labels. In *Proc. 27th International Joint Conference on Artificial Intelligence (IJCAI)*. 3033–3039. doi:10.24963/ijcai.2018/421

[46] Zhanhao Zhao, Haotian Gao, Naili Xing, Lingze Zeng, Meihui Zhang, Gang Chen, Manuel Rigger, and Beng Chin Ooi. 2025. NeurBench: A Benchmark Suite for Learned Database Components with Drift Modeling. *Proc. ACM Manag. Data* (2025). doi:10.1145/3802091

[47] Wei Zhou, Chen Lin, Xuanhe Zhou, and Guoliang Li. 2024. Breaking It Down: An In-Depth Study of Index Advisors. *Proc. VLDB Endowment* 17, 10 (2024), 2405–2418. doi:10.14778/3675034.3675035